

Homework 01: Professionalizing a Small Data Science Project

Contents

- Environments, Structure, Reproducibility, and Containerization
- Overview and Instructions
- Part 0: Clone and Setup (5 points)
- Part 1: Project Structure and .gitignore (15 points)
- Part 2: Environment Management (15 points)
- Part 3: README for Local Reproduction (15 points)
- Part 4: Personalize the analysis (15 points)
- Part 5: Containerize with Docker (15 points)
- Part 6: Extend README for Docker (5 points)
- Part 7: AI Audit Log (5 points)
- Part 8: Submission and Reflection (10 points)

Environments, Structure, Reproducibility, and Containerization

Due date: March 13, 2026 by 11:59 PM

Points: 100 total

Starter repo for this homework: [msu-cmse-courses/cmse492-hw01-starter](https://github.com/msu-cmse-courses/cmse492-hw01-starter)

Name (FIRST and LAST) — delete this text and type your name here before submitting this notebook.

Overview and Instructions

Welcome to Homework 01! The goal is to take a small data science project (provided in the starter repo above) and “professionalize” it so that **anyone** can clone your repo, set up the environment (local or Docker), and reproduce a key result (e.g., a summary table or plot) in under 30 minutes.

But the time you finish this assignment, you will have:

1. Cloned the starter repo and restructured it to meet professional standards.
2. Added environment support (choose venv, conda, or uv).
3. Written a clear README for local reproduction.
4. Personalized the analysis in some way.
5. Containerized it with Docker.
6. Extended README for Docker reproduction.
7. Created and AI audit log of your process
8. Reflected on the process and what you learned.
9. Pushed to **your own private GitHub repo** and submitted the link + reflection.

Work in your own private repo. Do **not** try to commit to the starter repo (!!).

Submission:

- GitHub repo URL (private; add instructor/TA as collaborators), provided via MS Form.
- Reflection (~300 words) at the end of this notebook, pasted into the MS Form.
- For completeness, this notebook with appropriate sections filled out will be submitted to D2L as well, but the main submission is via GitHub + MS Form.

Grading breakdown: See section headings for point values.

Note about AI tools: As discussed at the beginning of the course, you are allowed to use AI tools (e.g., ChatGPT, Copilot) in responsible, transparent, and ethical ways. For this particular assignment, if you end up using AI tools for assignment, but you will be expected to document your usage in an “AI Audit Log” section near the end of this notebook. See the details in the “AI Audit Log” section below.

Part 0: Clone and Setup (5 points)

To get started, you will be cloning a starter repo that contains a small data science project. This repo was created as a “messy” example that you might inherit from a classmate or colleague. The original version of the repo was constructed using the OpenAI Codex agent and might contain code that seems unnecessarily complex or illogical, which is not uncommon when folks use AI tools without careful oversight.

You are tasked with taking this messy starter repo and turn it into a well-organized, professional project that follows best practices for structure, environment management, reproducibility, and documentation.

Your first step is to clone this repo, set up your own private GitHub repo, and push the starter code to your repo:

1. Clone the starter repo to a local folder:

```
git clone https://github.com/msu-cmse-courses/cmse492-hw01-starter.git hw01-yourMSUNetID
cd hw01-yourMSUNetID
```

Note: make sure to replace `yourMSUNetID` with your actual MSU NetID to create a unique folder name for your project!!

2. Open the folder in VS Code.
3. On [GitHub](#), Create a **new private GitHub repo** named `cmse492-hw01-yourMSUNetID`
 - **Make sure to use your actual MSU NetID in place of** `yourMSUNetID`
 - Do not initialize with README, you should be able to use the repo creation defaults other than setting the repo name and making it private.
4. Add your cloned starter as a remote and push (replace `yourusername` and `yourMSUNetID` appropriately):

```
git remote add origin https://github.com/yourusername/cmse492-hw01-yourMSUNetID
git branch -M main
git push -u origin main
```

⚠ **Git Checkpoint:** Run `git status` and confirm you're on `main` with a clean working

tree.

Part 1: Project Structure and .gitignore (15 points)

The starter repo has no meaningful organization or structure.

Your job is to restructure it into a more professional format that separates core scripts, data, and results. This new layout should be intuitive and make it easy for others to find key components of your project with minimal effort. **You are not required to follow an exact structure** but should be able to justify your organizational choices in your reflection.

Tasks:

1. Create appropriate folders for organizing the repository content.
2. Move the provided files into these folders as warranted.
 - **Important:** After moving files, make sure to update any file paths in the code so that it still runs correctly! (This should be true for all `.py` and `.ipynb` files, but the exact files that need path updates will depend on how you reorganize the repo.)
3. Update `.gitignore` to include files that *should not be tracked* (e.g., virtual environments, caches, large data files, results, etc.)
 - **Make sure to test:** `git status` should show only your changes (no `.venv` or caches, etc.).

⚠ **Git Checkpoint:** Once you do the reorganization and update `.gitignore`, commit your changes. Make sure that all of the files you had to touch are included in the commit, and **use a meaningful commit message!**

Paste your `git status` output and folder tree (`tree` on Linux/Windows or `ls -aR` as an equivalent on Mac) below.

Side note for Mac users: you can install `tree` via Homebrew (`brew install tree`) to show the recursive folder structure in the same way it can be seen on Linux/Windows, but you're not required to do this.

Side note for Linux users: if you don't have `tree` installed, you can usually install it via your package manager (e.g., `sudo apt install tree` on Debian/Ubuntu).

Paste git status and folder structure here.

Run `git status` and `tree` or `ls -aR` in terminal, paste output.

Part 2: Environment Management (15 points)

Choose **at least two** of the possible paths below for setting up a virtual environment that can run the analysis. Implement each of them, test that your analysis runs, and generate the necessary environment file.

Option A: venv + requirements.txt

For this option, you'll set up a Python virtual environment using `venv`, install the necessary dependencies, and generate a `requirements.txt` file.

Once you have everything set up, test that your analysis runs successfully by activating the virtual environment and running the analysis script, `analysis.py`.

Option B: conda + environment.yml

For this option, you'll create a Conda environment, install the necessary dependencies, and generate an `environment.yml` file.

Once you have everything set up, test that your analysis runs successfully by activating the Conda environment and running the analysis script, `analysis.py`.

Option C: uv + pyproject.toml

For this option, you'll use `uv` to manage your Python environment and dependencies. You'll create a `pyproject.toml` file to specify your dependencies.

Once you have everything set up, test that your analysis runs successfully by using `uv` to run the analysis script, `analysis.py`.

Which two options did you implement? Paste your environment file contents for both methods below:

Paste contents of requirements.txt, environment.yml, or pyproject.toml here

⚠ **Git Checkpoint:** Commit your environment files. You should commit each option you implemented using separate commits with meaningful messages. This means “`git add`”-ing and committing each environment file separately with a message that indicates which method you implemented.

Part 3: README for Local Reproduction (15 points)

Edit `README.md` with these **exact sections** (use Markdown headers):

1. Overview

In 2–4 sentences explain the purpose of this project repo: What does it do? What data is used? What is the expected result? You may wish to include a visual representation of the repo structure here to orient users.

2. Data

Include details on location/format (e.g., “Small CSV in data/sample.csv”).

- Where does the data live, what’s the file format, and how is it structured? If it’s generated by a script, which one and how do you run it? If you need to download it from an external source, where is it and how do you get it?

3. Environment Setup

Include copy-paste commands for your chosen environment setup paths (there should be two!). If appropriate, include instructions for downloading/installing Python, Conda, or uv. The user should be able to copy-paste these commands to get set up without needing to read documentation or troubleshoot.

4. Reproduce a Result

Specify the exact commands the user should execute to reproduce the result. Clearly articulate what the expected output should be and how the user confirms that everything is working as intended.

5. Troubleshooting

Include potential OS-specific notes, common errors, and how to resolve them. This is

where you try to anticipate where users might get stuck and provide guidance to help them overcome those issues. You might have come across these yourself in the processing of setting everything up, so this is your chance to share that knowledge and make it easier for others.

Test it all out!

Pretend you're a stranger—can you clone the repo, follow your README instructions, and reproduce the result without peeking at the code or troubleshooting on Google? If not, revise your README and try again until you can.

Paste your [README.md](#) content below.

Paste [README.md](#) content here

Make sure to paste the final version that you tested for reproducibility, not an earlier draft.

⚠ **Git Checkpoint:** Commit your final [README.md](#) file to your repo with a meaningful commit message and push it to your repo.

Part 4: Personalize the analysis (15 points)

In order to show that you understand what the code is doing and how it all connects, you are now expected to modify it to **do something new/different** while ensuring that your new code is run when `analysis.py` is executed.

When doing your modification, you should aim to make a change that is meaningful and adds value to the analysis, but also one that can be implemented with minimal changes to the overall code structure. The goal is to demonstrate your understanding of the existing code and how to extend it in a way that is integrated and reproducible.

When making your modification(s), **make sure that:**

1. Your modification is integrated into the existing codebase and runs when `analysis.py` is executed. This means that you should not just create a separate stand-alone script or

notebook for your modification, but rather incorporate it into the existing workflow of the analysis. This could involve adding a new function to an existing script, modifying an existing function to include your changes, or adding a new step in the analysis pipeline that is executed when `analysis.py` is run, which could be a separate script that is called from `analysis.py`.

2. Your modification you should an **additional** Python package (i.e., one that is not already being used in the existing codebase). This will allow you to demonstrate your ability to integrate new tools and libraries into an existing project, which is a common task in real-world data science work. Make sure to add this new package to your environment files so that others can reproduce your analysis with the new modification.
3. Your modification should be apparent in the outputs produced when `analysis.py` is run. For example, if you add a new column to a summary table, that column should be visible in the output CSV file. If you create a new plot, it should be saved as an image file in the results folder. The idea is that someone who runs your analysis should be able to easily see the impact of your modification in the results.

Your personal modification might:

- Add a new component to the analysis (e.g., a new summary table or a new plot)
- Filter the data in a new way (e.g., focus on a specific subset of the data)
- Compute a new metric or transformation (e.g., a new summary statistic)

Once you are happy with your modifications, **make sure that your environment can still run the analysis and generate the new outputs without errors**. This is a key part of the exercise: ensuring that your personalized changes are integrated into the existing codebase and can be executed smoothly.

As needed, **update your environment files and README instructions** to reflect any new dependencies or changes in the analysis process.

Within your README, **you should also briefly describe what your personal modification is and how it changes the analysis**. Add this as a new section in your README (e.g., "Personal Analysis") where you explain what you did and how it impacts the results.

Describe your modification and its impact on the analysis below. If helpful, paste before/after results/outputs as well.

Describe your personal modification and its impact on the analysis

Paste result/output content as appropriate.

⚠️ **Git Checkpoint:** Commit your personalized analysis changes with a meaningful commit message that describes what you did. **Make sure your modifications to your README and environment files are also included in this commit!**

Part 5: Containerize with Docker (15 points)

For this section, you're going to create a Dockerfile that sets up an environment with all the necessary dependencies and runs the analysis script to generate the results. This will allow anyone to reproduce your analysis in a consistent environment without worrying about local setup.

Prerequisites for completing this section: Docker installed (`docker --version` && `docker run --rm hello-world` should both return successful outputs).

1. Create the Dockerfile

Create a `Dockerfile` in the repo root directory. The Dockerfile should

- Use an appropriate base image for Python analysis (you can use one that we used in class or find one that suits your needs, see [Docker Hub](#) for options).
- Optional: set environment variables to prevent Python from writing bytecode and to ensure unbuffered output.
- Set the working directory to `/app`.
- Install the necessary dependencies using the environment file(s) you created (e.g., `requirements.txt`).
- Copy the project files into the container.
- Run the analysis script to generate the results.

2. Build & Test

Once you have the Dockerfile set up, you can build the Docker image and run it to test that everything works as expected. You should be able to run the analysis and generate the new outputs without any issues.

From with your root directory of your repository you should be able to run `docker build` and `docker run` commands to build the image and execute the analysis. If needed, make sure to mount the appropriate volumes so that the data and results are accessible outside the container (see the Day 09 materials for examples of how to do this).

Paste your Dockerfile content, the commands need to build and run Docker, and the outputs of `docker build` and `docker run` below.

Paste Dockerfile content here

Make sure to paste the final version that you are confident works, not an earlier draft.

Paste command(s) to build and run Docker here

Make sure these should work from the root of your repo and that they are the final versions you tested, not earlier drafts.

Paste terminal output of docker build here:

Just include the first 10–20 lines and the success message at the end, not the entire log.

Paste terminal output of docker run here:

This should match the expected output you defined in your README.

 **Git Checkpoint:** Commit your Dockerfile with a meaningful commit message and push it to your repo.

Part 6: Extend README for Docker (5 points)

Now that you have Docker as a working option for running your analysis, you need to update your README to include instructions for how to use Docker to reproduce the results.

This should be a new section in your README (e.g., “Containerized Run”) that provides clear, step-by-step instructions for building the Docker image and running the container to execute the analysis.

You should also provide the steps need to set up Docker if the user doesn't have it installed, and any OS-specific notes for running Docker commands (remember, some of the commands for mounting volumes differ between Windows and macOS/Linux).

Finally, there should be details for what the user should expect to see when they run the Docker container and how they can confirm that it worked correctly.

Paste updated README section below.

Paste updated README section below.

This should *not* be the entire README, just the new section you added for Docker instructions.

 **Git Checkpoint:** Commit your updated README with a meaningful commit message and push it to your repo.

Part 7: AI Audit Log (5 points)

As mentioned above, you are allowed to use AI tools in the course and for this assignment, but you **must document** this usage in the table below. If you used AI tools but did not document them here, that would be considered an integrity violation and does not align with your responsibility as a professional data scientist to be transparent about your process and the tools you use.

If you did not use any AI tools, simply state "No AI tools used" in the table below and leave the rest of the columns blank.

Remember, there is no inherent penalty for using AI tools, so transparency is valued here. The goal of this table is to encourage you to be mindful and intentional about how you use AI tools, and to reflect on their role in your workflow.

Fill out the table accurately and honestly (the first row is provided as an example):

Assignment Section	AI Tool Used	Prompt(s) used	AI-generated Output	YOUR Changes/Why	Did you verify the output? How?
Part 5.1, Creating Dockerfile	Claude	"Create a dockerfile that can be used to run a python data analysis script from inside a project repo using a uv-managed python environment"	Generated a Dockerfile template	Adjusted the python image choice, removed uv installation step, modified the directory path to match my setup	Yes, by running <code>docker build</code> and verifying the build was successful and aligned with the expected assignment outcomes.
...	...	...	...	...	...

Add to the above table to document the role AI played in your work.

Part 8. Submission and Reflection (10 points)

1. **Repo URL:** [Paste your private GitHub repo URL here]
2. **Reflection (~300 words):** Take a moment to reflect on everything you did to "professionalize" and personalize the original repository and the choices you made. Make sure your reflection addresses the following points:
 - What changes made it easier to understand and more reproducible (incl. Docker)?
 - Which env tool(s) did you use and why? How does Docker complement this?
 - What changes did you make to personalize the analysis and why? How do these changes demonstrate your understanding of the code and the analysis process?
 - What concerns might you have if you were handing to an intern or employer?

This reflection should be thoughtful and demonstrate your understanding of the process you went through to professionalize the repository, **in your own words**. This should be seen as a low-stakes opportunity to articulate what you learned, the challenges you faced, and how you overcame them. Be honest about any difficulties you

encountered and how you resolved them, as this is a key part of the learning process. There is no “right” answer here, so focus on being clear and reflective rather than trying to crafting the perfect response.

This reflection will be submitted via the MS Form below, but you can write it here first to get it down and then copy-paste it into the form when you’re ready to submit.

This reflection is not being graded for grammar or style, so don’t worry about making it perfect.

Paste your ~300-word reflection here.

Example start: In this homework, I restructured the project into src/data/results for better organization...’’

Submit this notebook to D2L and Fill out the MS Form below to finalize your submission. (If the form won’t render in your notebook for some reason, you can also access it directly here: <https://forms.office.com/r/ay9bpLqx3R>)

```
from IPython.display import HTML
HTML(
    """
    <iframe
        src="https://forms.office.com/r/ay9bpLqx3R"
        width="800"
        height="800px"
        frameborder="0"
        marginheight="0"
        marginwidth="0">
        Loading...
    </iframe>
    """
)
```

Congratulations, you’re done!

Submit this assignment by uploading it to the course Desire2Learn web page. Go to the “Homework” section, find the appropriate submission link, and upload it there. **Make sure your instructors have access to your private GitHub repo by adding them as collaborators!**

See you in class!

© Copyright 2026, The Department of Computational Mathematics, Science and Engineering at Michigan State University.